

개발자형 코드 블로그 프로젝트 기획서

1. 프로젝트 개요

1.1 프로젝트 명

DevCodeBlog - 코드 작성, 실행 결과, 메모를 통합 관리하는 개발자 블로그

1.2 프로젝트 목적

- 개발자가 코드 작성 과정과 실행 결과, 학습 메모를 하나의 플랫폼에서 체계적으로 관리
- 코드 스니펫과 설명을 효과적으로 공유하고 기록할 수 있는 개인 블로그 구축
- 학습한 내용과 문제 해결 과정을 시각적으로 정리하여 포트폴리오 역할 수행

1.3 타겟 사용자

- 개인 개발자 (1차 타겟)
- 프로그래밍 학습자
- 기술 블로그 운영을 원하는 개발자

2. 핵심 기능 요구사항

2.1 코드 작성 기능

- 코드 에디터: 문법 하이라이트 지원 에디터 (Monaco Editor 활용)
- 다중 언어 지원: JavaScript, Python, Java, SQL 등
- 코드 저장 및 버전 관리: 작성한 코드의 히스토리 관리
- 실시간 미리보기: 작성 중인 코드 실시간 확인

2.2 코드 실행 및 뷰 기능

- 코드 실행: 간단한 코드 실행 (클라이언트 사이드 또는 샌드박스)
- 실행 결과 표시: 콘솔 출력, 오류 메시지 등 결과 화면
- 코드 공유: 작성한 코드의 공유 링크 생성
- 문법 하이라이트: 저장된 코드의 가독성 있는 표시

2.3 메모 및 설명 기능

- 마크다운 에디터: 코드 설명을 위한 마크다운 지원
- 코드-메모 연동: 특정 코드 블록과 메모 연결
- 태그 시스템: 코드와 메모의 분류 및 검색 지원
- 첨부 파일: 이미지, 문서 등 첨부 기능

2.4 블로그 기본 기능

- 포스트 관리: 생성, 수정, 삭제, 공개/비공개 설정
- 카테고리 분류: 언어별, 주제별 분류
- 검색 기능: 제목, 내용, 태그 기반 검색
- 댓글 시스템: 포스트별 댓글 (선택사항)

3. 사용자 시나리오

3.1 메인 시나리오

1. 포스트 작성

- 사용자가 새 포스트 생성
- 제목, 카테고리, 태그 설정
- 코드 에디터에서 코드 작성
- 실행 버튼으로 코드 테스트
- 마크다운 에디터로 설명 작성
- 포스트 저장 및 공개

2. 포스트 조회

- 블로그 메인 페이지에서 포스트 목록 확인
- 특정 포스트 클릭하여 상세 보기
- 코드 블록과 설명을 함께 확인
- 필요시 코드 복사 또는 실행

3. 관리 기능

- 대시보드에서 작성한 포스트 관리
- 카테고리별, 태그별 포스트 분류
- 통계 정보 (조회수, 인기 포스트 등)

4. 기술 스택

4.1 백엔드

- **Framework:** Spring Boot (Java)
- **Database:** MariaDB
- **Authentication:** JWT 토큰 기반 인증
- **File Storage:** 로컬 파일 시스템 또는 클라우드 스토리지
- **API:** RESTful API 설계

4.2 프론트엔드

- **Framework:** React.js
- **State Management:** Redux Toolkit 또는 Context API
- **Code Editor:** Monaco Editor (VS Code 에디터)
- **Markdown:** React-Markdown
- **Styling:** Tailwind CSS 또는 Material-UI
- **Build Tool:** Vite 또는 Create React App

4.3 개발 환경

- **Version Control:** Git/GitHub
- **Development:** Docker (선택사항)
- **Deployment:** 단일 서버 배포 (Nginx + Spring Boot)

5. 데이터베이스 설계

5.1 주요 테이블

```
-- 사용자 테이블
users (id, username, email, password_hash, created_at)

-- 포스트 테이블
posts (id, user_id, title, content, code_content, language,
       category, tags, is_public, created_at, updated_at)

-- 카테고리 테이블
categories (id, name, description)

-- 태그 테이블
tags (id, name)

-- 포스트-태그 관계 테이블
post_tags (post_id, tag_id)
```

```
-- 코멘트 테이블 (선택사항)
comments (id, post_id, author_name, content, created_at)
```

6. UI/UX 주요 화면

6.1 메인 페이지

- 헤더: 로고, 네비게이션, 로그인/로그아웃
- 사이드바: 카테고리, 태그 클라우드, 최신 포스트
- 메인 콘텐츠: 포스트 목록 (카드 형태)
- 푸터: 저작권, 링크

6.2 포스트 작성 페이지

- 좌측: 코드 에디터 (Monaco Editor)
- 우측 상단: 실행 결과 창
- 우측 하단: 마크다운 에디터
- 상단: 제목, 카테고리, 태그 입력
- 하단: 저장, 공개, 미리보기 버튼

6.3 포스트 상세 페이지

- 상단: 제목, 작성일, 카테고리, 태그
- 메인: 코드 블록과 설명이 번갈아 표시
- 코드 블록: 복사 버튼, 실행 버튼 (가능한 경우)
- 하단: 이전/다음 포스트, 댓글 (선택사항)

7. 개발 일정

7.1 1단계 (4주) - 기본 구조 및 핵심 기능

- Week 1: 프로젝트 셋업, DB 설계, 기본 API 개발
- Week 2: 사용자 인증, 포스트 CRUD API
- Week 3: 프론트엔드 기본 구조, 메인 페이지
- Week 4: 포스트 작성/조회 페이지, 코드 에디터 연동

7.2 2단계 (3주) - 고급 기능

- Week 5: 코드 실행 기능, 파일 업로드
- Week 6: 검색 기능, 카테고리/태그 관리
- Week 7: UI/UX 개선, 반응형 디자인

7.3 3단계 (2주) - 배포 및 최적화

- Week 8: 테스트, 버그 수정
- Week 9: 배포, 성능 최적화

8. 향후 확장 계획

8.1 중기 계획

- 다중 사용자 지원 (팀 블로그)
- 소셜 기능 (좋아요, 팔로우)
- 코드 실행 환경 확장 (Docker 기반)
- 모바일 앱 개발

8.2 장기 계획

- AI 기반 코드 분석 및 추천
- 온라인 코드 실행 환경 (Judge 시스템)
- 협업 기능 (코드 리뷰, 공동 작성)
- API 공개 및 플러그인 시스템

9. 기술적 고려사항

9.1 보안

- 코드 실행 시 보안 샌드박스 적용
- XSS, CSRF 공격 방지
- 업로드 파일 검증

9.2 성능

- 코드 하이라이트 라이브러리 최적화
- 이미지 최적화 및 CDN 활용
- 데이터베이스 쿼리 최적화

9.3 확장성

- 모듈식 아키텍처 설계
- API 버전 관리
- 캐싱 전략 (Redis 고려)

10. 성공 지표

10.1 기능적 지표

- 포스트 작성/조회 기능 정상 작동
- 코드 실행 성공률 95% 이상
- 페이지 로딩 시간 3초 이내

10.2 사용성 지표

- 사용자 만족도 조사
- 일일 활성 사용자 (본인 사용 기준)
- 포스트 작성 빈도

이 기획서를 바탕으로 단계별 개발을 진행하여 효율적이고 실용적인 개발자 블로그를 구축할 수 있습니다.